Topic 8: Functions

Task 1: Marks and grades toolkit

```
func average(of marks: Int...) -> Double {
    if marks.isEmpty {
        return 0
    }
    var total = 0
    for mark in marks {
        total += mark
    }
    return Double(total) / Double(marks.count)
}
func grade(for average: Double) -> String {
    if average >= 90 {
        return "O"
    } else if average >= 80 {
        return "A+"
    } else if average >= 70 {
        return "A"
    } else if average >= 60 {
        return "B"
    } else {
        return "RA"
    }
}
func stats(of marks: [Int]) -> (min: Int, max: Int, average: Double)? {
    if marks.isEmpty {
        return nil
```

```
    }
    var smallest = marks[0]
    var largest = marks[0]
    var total = 0
    for mark in marks {
        if mark < smallest { smallest = mark }
        if mark > largest { largest = mark }
        total += mark
    }
    let avg = Double(total) / Double(marks.count)
    return (min: smallest, max: largest, average: avg)
}
func curve(_ marks: inout [Int], by bonus: Int = 5) {
    for i in 0..<marks.count {
        marks[i] = min(marks[i] + bonus, 100)
    }
}
var marks = [85, 92, 67, 98, 74]
let avg = average(of: 85, 92, 67, 98, 74)
print("Average: \(avg)")
print("Grade: \(grade(for: avg))")
print("No marks average: \(average())")
if let result = stats(of: marks) {
    print("Min: \(result.min), Max: \(result.max), Average: \(result.average)")
}

let emptyMarks: [Int] = []
if let result = stats(of: emptyMarks) {
    print("Average: \(result.average)")
```

```
} else {
    print("No marks to analyse")
}
curve(&marks)
print("After curve: \(marks)")
```

Output

```
Average: 83.2
Grade: A+
No marks average: 0.0
Min: 67, Max: 98, Average: 83.2
No marks to analyse
After curve: [90, 97, 72, 100, 79]
```

Task 2: Text utilities

```
func isPalindrome(_ text: String) -> Bool {
    var cleaned = ""
    for character in text.lowercased() {
        if character.isLetter || character.isNumber {
            cleaned.append(character)
        }
    }
    return cleaned == String(cleaned.reversed())
}
func vowelCount(in text: String) -> Int {
    var count = 0
    for character in text.lowercased() {
        if "aeiou".contains(character) {
            count += 1
```

```
        }
    }
    return count
}
func wordCount(in text: String) -> Int {
    return text.split(separator: " ").count
}
func wordCount(in text: String, minLength: Int) -> Int {
    var count = 0
    for word in text.split(separator: " ") {
        var letters = 0
        for character in word {
            if character.isLetter {
                letters += 1
            }
        }
        if letters >= minLength {
            count += 1
        }
    }
    return count
}
let sentence = "A man, a plan, a canal: Panama"
print("Palindrome? \(isPalindrome(sentence))")
print("Vowels: \(vowelCount(in: sentence))")
print("Words: \(wordCount(in: sentence))")
print("Words with 4+ letters: \(wordCount(in: sentence, minLength: 4))")
```

Output

```
Palindrome? true
Vowels: 10
Words: 7
Words with 4+ letters: 3
```

Task 3: Functions as values

```
func makeMultiplier(_ factor: Int) -> (Int) -> Int {
    return { number in
        return number * factor
    }
}
let double = makeMultiplier(2)
let triple = makeMultiplier(3)
print(double(5))
print(triple(5))
func apply(_ values: [Int], using transform: (Int) -> Int) -> [Int] {
    var result: [Int] = []
    for value in values {
        result.append(transform(value))
    }
    return result
}
let numbers = [1, 2, 3, 4, 5]
print(apply(numbers, using: double))
print(apply(numbers, using: triple))
let squares = apply(numbers, using: { number in
```

```
    return number * number
})
print(squares)
```

```
10
15
[2, 4, 6, 8, 10]
[3, 6, 9, 12, 15]
[1, 4, 9, 16, 25]
```